Computer Architecture Lab 08 Report

张兆飞

2026-05-15

实验 8 Pipeline 实验

1. 实验目的

在 SMART 平台上通过调整分支预测配置情况观察测试程序程序的 CPI, 分支预测准确率等, 了解各种配置对 CPU 的性能的影响.

2. 实验步骤

2.1. 更改以及替换 SMART 平台内对应的文件, 包括 `crt0.s`, `dhrystone` 程序和 `coremark` 程序

为实现完全自动化测试, 本实验编写了 `run_bp_experiments.csh` C Shell Script 自动完成环境初始化, 文件备份, 核心代码替换以及后续仿真任务.

```
#!/bin/csh

# Lab 8: C910 branch-prediction sweep — 4 MHCR configs x 2 benchmarks = 8 sims.

# Paths
if ( ! $?SMART_DIR )   setenv SMART_DIR   "$HOME/zfzhang_24301050026/smart9_release"
if ( ! $?LAB_SHARE )   setenv LAB_SHARE   "/home/ECDesign/ECDesign_share/lab8"
set RESULTS_DIR = "$cwd/bp_results"
set EDIT_PY     = "$cwd/edit_crt0.py"
set CRT0        = "$SMART_DIR/lib/crt0.s"
set WORK_DIR    = "$SMART_DIR/workdir"
set LOG_FILE    = "$WORK_DIR/run.log"
set POLL_SEC    = 30

mkdir -p "$RESULTS_DIR"
echo "[setup] SMART_DIR = $SMART_DIR"
echo "[setup] RESULTS   = $RESULTS_DIR"

# Step 1: install lab-provided files (idempotent)
rm -f "$SMART_DIR/lib/crt0.s" "$SMART_DIR/case/dhry/Main.c" "$SMART_DIR/case/coremark/core_main.c"
cp -f "$LAB_SHARE/crt0.s"      "$SMART_DIR/lib/crt0.s"
cp -f "$LAB_SHARE/Main.c"      "$SMART_DIR/case/dhry/Main.c"
cp -f "$LAB_SHARE/core_main.c" "$SMART_DIR/case/coremark/core_main.c"
rm -f "$RESULTS_DIR/crt0.s.orig" >& /dev/null
cp -f "$CRT0" "$RESULTS_DIR/crt0.s.orig"   # backup pristine

# Step 2: source SMART env
cd "$SMART_DIR"/..
source cshrc >& /dev/null
cd "$SMART_DIR"
source setup.csh >& /dev/null
```

```
# Step 3: 4 configs x 2 benchmarks
foreach cfg ( all_on all_off btb_off bpe_off )
    echo ""
    echo "=================================================="
    echo "[cfg ] $cfg"
    echo "=================================================="
    python3 "$EDIT_PY" "$CRT0" "$cfg"

    foreach bench ( dhrystone coremark )
        if ( "$bench" == "dhrystone" ) then
            set src = "../case/dhry/Main.c"
        else
            set src = "../case/coremark/core_main.c"
        endif
        set out_dir = "$RESULTS_DIR/${bench}_${cfg}"
        mkdir -p "$out_dir"
        echo "[run ] cfg=$cfg  bench=$bench"

        cd "$WORK_DIR"
        if ( -e "$LOG_FILE" ) rm -f "$LOG_FILE"
        run "$src" >& /dev/null

        # poll for completion marker
        set is_done = 0
        while ( $is_done == 0 )
            sleep $POLL_SEC
            if ( -e "$LOG_FILE" ) then
                grep "simulation finished successfully" "$LOG_FILE" >& /dev/null
                if ( $status == 0 ) set is_done = 1
            endif
        end

        rm -f "$out_dir/run.log" "$out_dir/crt0.s.used" >& /dev/null
        cp -f "$LOG_FILE" "$out_dir/run.log"
        cp -f "$CRT0"     "$out_dir/crt0.s.used"
        echo "[done] -> $out_dir/run.log"
    end
end

# Step 4: restore pristine crt0.s
rm -f "$CRT0" >& /dev/null
cp -f "$RESULTS_DIR/crt0.s.orig" "$CRT0"

echo ""
echo "=================================================="
echo "[ok] all 8 runs complete."
echo "    parse:  python3 parse_bp_results.py $RESULTS_DIR"
echo "=================================================="
```

2.2. 在启动文件 `crt0.s` 中选择分支预测的配置, 并进行 `dhrystone` 程序和 `coremark` 程序的仿真

通过修改 `crt0.s` 中对机器模式硬件配置寄存器的写入值, 依次激活 4 种处理器状态.

配置	状态
`0x10f7`	开启所有预测器 (all prediction on)
`0x0007`	关闭所有预测器 (all prediction off)

配置	状态
0x00b7	仅关闭 BTB 与 L0BTB (BTB,L0BTB off)
0x10d7	仅关闭 BHT, 即关闭分支预测使能 (BPE off)

为避免手动修改汇编代码带来的错误, 使用 `edit_crt0.py` Python Script 进行修改.

```python
#!/usr/bin/env python3

"""Toggle which MHCR-write line is active in crt0.s.
Identifies the 4 candidate lines by their unique hex constants."""

import sys, re, pathlib

HEX = {
    'all_on':  '10f7',   # all prediction on
    'all_off': '0007',   # all prediction off
    'btb_off': '00b7',   # BTB,L0BTB off
    'bpe_off': '10d7',   # BPE off (BHT off)
}
path, choice = sys.argv[1], sys.argv[2]
target = HEX[choice]

cand_re = re.compile(r'^(\s*)#?\s*(li\s+x3\s*,\s*0x([0-9a-fA-F]+))(.*)$')
out = []
for ln in pathlib.Path(path).read_text().splitlines():
    m = cand_re.match(ln)
    if m and m.group(3).lower() in HEX.values():
        indent, instr, hexval, rest = m.group(1), m.group(2), m.group(3).lower(), m.group(4)
        prefix = '' if hexval == target else '#'
        out.append(f'{indent}{prefix}{instr}{rest}')
    else:
        out.append(ln)

pathlib.Path(path).write_text('\n'.join(out) + '\n')
print(f"[edit_crt0] {choice}: enabled 0x{target}, others commented")
```

2.3. 完成分支预测配置下的 `dhrystone` 程序和 `coremark` 程序的仿真后, 观察仿真结果, 记录数据, 汇总成上述的两张表格

仿真结束后, 利用 `parse_bp_results.py` Python Script 自动扫描 `run.log` 日志文件, 提取各项性能指标并计算 CPI.

```python
#!/usr/bin/env python3

"""Parse run.log into the two Lab 8 tables."""

import sys, re, pathlib, csv

RESULTS = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "bp_results")
CONFIGS = ["all_on", "all_off", "btb_off", "bpe_off"]
LABEL = {
    "all_on":  "all prediction on",
    "all_off": "all prediction off",
    "btb_off": "BTB,L0BTB off",
    "bpe_off": "BPE off",
}
```

```
def gv(text, var):
    m = re.search(rf'num_{var}\s+is\s+(-?\d+)', text)
    return int(m.group(1)) if m else None

def parse(log_path):
    if not log_path.exists():
        return {}
    txt = log_path.read_text(errors="ignore")
    cycle = gv(txt, "cycle")
    insts = gv(txt, "instret")
    row = {
        "cycle":      cycle,
        "insts":      insts,
        "cpi":        (cycle / insts) if (cycle and insts) else None,
        "cond_miss":  gv(txt, "conditional_branch_mis"),
        "indir_miss": gv(txt, "indirect_branch_mis"),
        "indir_inst": gv(txt, "indirect_branch_inst"),
    }
    m = re.search(r'dhrystone\s+is\s+([\d.]+)\s+dmips/MHz', txt, re.I)
    if m: row["dmips_per_mhz"] = float(m.group(1))
    m = re.search(r'CoreMark\s*1\.0\s*:\s*([\d.]+)\s*CoreMark/MHz', txt, re.I)
    if not m:
        m = re.search(r'CoreMark/MHz\s*:\s*([\d.]+)', txt)
    if m: row["coremark_per_mhz"] = float(m.group(1))
    return row

def fmt(v):
    if v is None: return "-"
    if isinstance(v, float): return f"{v:.4f}"
    return str(v)

def build(bench, score_key, score_label, csv_name):
    rows = {c: parse(RESULTS / f"{bench}_{c}" / "run.log") for c in CONFIGS}
    metrics = [
        ("cycle", "cycle"), ("insts", "insts"), ("cpi", "CPI"),
        ("cond_miss",  "conditional branch miss"),
        ("indir_miss", "indirect branch miss"),
        ("indir_inst", "indirect branch inst"),
        (score_key, score_label),
    ]
    print(f"\n## {bench}\n")
    print("| metric | " + " | ".join(LABEL[c] for c in CONFIGS) + " |")
    print("|---" * (len(CONFIGS) + 1) + "|")
    for k, lbl in metrics:
        print(f"| {lbl} | " + " | ".join(fmt(rows[c].get(k)) for c in CONFIGS) + " |")
    csv_path = RESULTS / csv_name
    with csv_path.open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["metric"] + [LABEL[c] for c in CONFIGS])
        for k, lbl in metrics:
            w.writerow([lbl] + [fmt(rows[c].get(k)) for c in CONFIGS])
    print(f"\n  csv -> {csv_path}")

build("dhrystone", "dmips_per_mhz",    "DMPIS (dmips/MHz)",  "table1_dhrystone.csv")
build("coremark",  "coremark_per_mhz", "CoreMark/MHz",        "table2_coremark.csv")
```

脚本自动解析后生成的数据汇总如下.

dhrystone 测试指标	all prediction on	all prediction off	BTB,L0BTB off	BPE off
cycle	1,140,992	3,040,575	1,385,950	2,278,724
insts	2,040,028	2,040,028	2,040,028	2,040,028
CPI	0.5593	1.4905	0.6794	1.117
conditional branch miss	19	69,999	19	69,999
indirect branch miss	0	0	0	0
indirect branch inst	0	0	0	0
DMPIS (dmips/MHz)	4.9912	1.8717	4.1232	2.5066

coremark 测试指标	all prediction on	all prediction off	BTB,L0BTB off	BPE off
cycle	5,684,994	12,875,906	7,050,750	12,568,284
insts	9,474,454	9,474,454	9,474,454	9,474,454
CPI	0.6	1.359	0.7442	1.3265
conditional branch miss	59,995	682,829	62,141	682,829
indirect branch miss	5	320	5	5
indirect branch inst	320	320	320	320
CoreMark point (CoreMark/MHz)	7.0361	3.1066	5.6732	3.1826

3. 实验分析与总结

在预测全开时, 两个基准测试的 CPI 均远低于 1, 此时前端流水线处于充分供给状态. 一旦关闭 BPE, 两个测试程序的 CPI 迅速下降. 通过将增加的总周期数分摊到增加的分支预测失败次数上, 可以推算出每次分支预测失败带来的硬件惩罚约为 11 个周期, 这一估值与 C910 从前端取指到后端执行的真实流水线深度高度吻合.

对比“关闭所有预测”与“仅关闭 BHT”两种状态, 条件分支缺失次数在同一基准测试下完全一致. 这印证了 BHT 对分支预测缺失与否起决定性作用. 相较于预测全开, 仅关闭 BTB 时分支缺失次数几乎没有增加, 但总耗费周期却有较大幅度上升. 这表明 BTB 的意义在于, 当 BHT 成功预测为“跳转”时, 为处理器提供正确的目标地址.

在本次采用的特定测试中, 间接分支预测器的影响微乎其微. Dhrystone 完全没有间接分支的预测, 而 CoreMark 中总的间接分支指令也仅有 320 条. 尽管开启 IBPE 后能将此类指令的预测准确率提升至接近完美, 但由于其指令绝对基数极小, 对整体性能的提升几乎可以忽略不计.

4. 实验收获与建议

打破了将分支预测视为单一功能的刻板印象, 深入理解了现代微架构中如何将预测机制拆分为方向 (BHT), 目标 (BTB) 和间接跳转 (IBP) 等多个模块, 以及它们各自对性能的贡献. 通过实际修改汇编代码并观察性能指标的变化, 深刻体会到了分支预测技术在提升指令级并行性和处理器整体性能方面的关键作用, 为后续学习更复杂的设计奠定了坚实的基础.

建议未来的实验中减少不必要的重复劳动, 以便将更多精力集中在分析和理解上.